西南财经大学本科期末考试试题册（B）

（2023－2024学年第1学期）

以下各项由命题教师填写：

课程名称：数据结构（英）**命题教师：**陈中普 王海林

适用对象（年级专业）：2022级信息管理与信息系统、2022级物流管理、2022级金融数学

使用试题的任课教师姓名：陈中普 王海林

试题说明：

1、考试类型：闭卷[√] 开卷[ ]

2、本套试题共 四 道大题，共 5 页，完卷时间 120 分钟。

3、考试用品中除纸、笔、尺子外，可另带的用具有：

计算器[ ] 字典[ ] 等

（请在下划线上填上具体数字或内容，所选[ ]内打钩）

考试时间（由制卷方填写）：

以下各项由学生填写：

任课教师： **年级专业：**

学生姓名： **学 号：**

考生注意事项：1.出示学生证（或身份证）和准考证于桌面左上角，以备查验。

2.拿到试卷后清点并检查试卷页数，如有重页、页数不足、空白页及印刷模糊等举手向监考教师示意调换试卷。

3.答题前先将试题册及答题纸上的任课教师、专业、年级、学号、姓名填写完整。

4.所有答案均需填写在答题纸上，答在试题册上无效。

5.考生不得携带任何通讯工具进入考场。

6.考试结束后，将试题册、答题纸和草稿纸等下发材料上交监考教师，不得带离考场。

7.严格遵守考场纪律。

- We assume that the height of an empty tree is -1.
- The Node class in a singly linked list has attribute *data* and *next*, in which *data* is the element it is holding, and *next* is the link to the next node. If it is doubly linked list, the node holds *prev* link to the previous nodes.
- The Node class in a binary tree has attribute *key*, *left*, and *right*, in which *key* is the data it is holding, and *left* and *right* are links to its left and right child, respectively.

Part One: Multi-choices (2 marks each; 20 marks in total)

(*Note: Each question has only one correct choice.*)

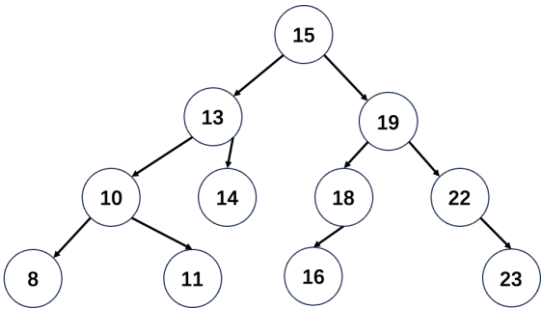
1. What values are returned during the following series of stack operations, if the stack is initially an empty stack? push(3), push(4), pop(), push(8), push(9), pop(), push(10), pop(), pop()
   A. 10, 9, 4, 8
   B. 3, 8, 10, 9
   C. 4, 9, 10, 8
   D. 8, 9, 4, 10

2. What values are returned during the following series of queue operations, if executed on an initially empty queue? enqueue(6), dequeue(), enqueue(3), enqueue(5), enqueue(8), dequeue(), enqueue(9), dequeue(), dequeue()
   A. 3, 6, 5, 8
   B. 3, 8, 9, 10
   C. 6, 3, 5, 8
   D. 9, 8, 5, 3

3. What is the time complexity of a LinkedList to remove the second to last element, if the LinkedList without the tail reference?
   A. O(1)
   B. O(N)
   C. $O(N^2)$
   D. O(*log*N)

4. Which of the following statement is correct in terms of the time complexity?
   A. $O(N + 3N^2)$ can often be simplified as O(N).
   B. Given a singly linked list represented by a *head* node, the time complexity of removing the tail node is O(1) in the worst case.
   C. Given a doubly linked list represented by a *head* node and another *node* pointer, the time complexity of adding a new element before *node* position is O(1) in the worst case.
   D. Given a BST storing integers, the time complexity of searching a given number is O(N) in the worst case.

5. Given a BST as follow, how many times of comparisons are required to search 11?



A. 4
B. 5
C. 3
D. 8

6. Given a complete binary tree whose height is 4, how many nodes can be there at most?
   A. 31
   B. 32
   C. 33
   D. 34

7. Which way can be used to deal with hashing collisions? i. Separate Chaining, ii. Open Addressing, iii. Linear Probing
   A. i, and ii
   B. i, ii, and iii
   C. ii, and iii
   D. i, and iii

8. Given the BST in Question 5, what is the pre-order tree walk?
   A. 8 -> 10 -> 11 -> 13 -> 14 -> 15 -> 16 -> 18 -> 19 -> 22 ->23
   B. 8 -> 11 -> 10 -> 14 -> 13 -> 16 -> 18 -> 23 -> 22 ->19 -> 15
   C. 15 -> 10 -> 13 -> 8 -> 11 -> 14 -> 19 -> 18 -> 16 -> 22 -> 23
   D. 15 -> 13 -> 10 -> 8 -> 11 -> 14 -> 19 -> 18 -> 16 -> 22 -> 23

9. Give a min-heap stored in an array [6, 8, 9, 20, 23, 10, 15, 24, 22], after removing 6, which is the correct array storing the min-heap?
   A. [8, 9, 20, 23, 10, 15, 24, 22]
   B. [9, 8, 20, 23, 10, 15, 24, 22]
   C. [8, 20, 9, 22, 23, 10, 15, 24]
   D. [8, 20, 9, 23, 22, 10, 15, 24]

10. Suppose Bob hashes [2, 4, 5, 9, 15, 10, 3] using separate chaining method where the hash function is $h(key) = key\%7$, and each chain is represented as a linked list. To check whether it contains 3, how many times of comparisons are required?

A. 1

B. 2
C. 3
D. 4

Part Two: True or False (2 mark each; 20 marks in total)

1. A linked list is a data structure that allows efficient insertion and deletion of elements at the end of the list. (True/False)
2. A deque is a data structure that allows efficient insertion and deletion of elements at both ends. (True/False)
3. A binary search tree is a data structure that allows efficient search, insertion, and deletion of elements in logarithmic time. (True/False)
4. A graph is a data structure that represents relationships between entities using edges and nodes. (True/False)
5. A circular linked list is a data structure that allows efficient insertion and deletion of elements at the end of the list but has a loop in it. (True/False)
6. A doubly linked list is a data structure that allows efficient access to elements using both previous and next pointers. (True/False)
7. A priority queue is a data structure that allows efficient retrieval of the element with the highest priority. (True/False)
8. A linked list is more suitable for insertion and deletion operations, while an array is more suitable for random access operations. (True/False)
9. A hash table provides faster search time compared to a binary search tree when the number of elements is large. (True/False)
10. In a binary search tree, the time complexity of search, insertion, and deletion operations is O(n), where n is the number of elements in the tree. (True/False)

Part Three: Short Q&A (6 marks each; 24 marks in total)

（**Note that only key points are required and try to keep your answers short**）

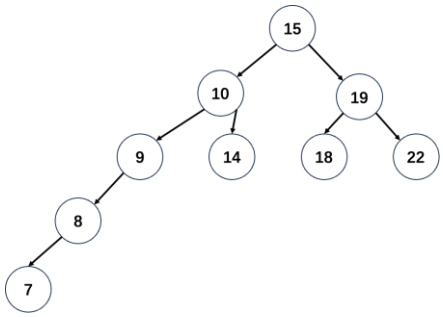
1. If $h(n) = (n+1)^2 + (n+2)^2$, is it true that $h(n) = O(n^2)$? Explain your answer.
2. In a doubly linked list, there are two special nodes at the beginning and end of the list: the **sentinel** nodes. The first sentinel node is placed at the start of the list, and its *next* pointer points to the first data node. The second sentinel node is placed at the end of the list, and its *prev* pointer points to the last data node. Explain their advantages and disadvantages.
3. The following code is supposed to implement a stack using nodes of a linked list. However, there is an error in the code that prevents it from working correctly. Please identify the bug and provide the corrected code.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
class Stack:
    def __init__(self):
        self.head = None

    def push(self, data):
        new_node = Node()
        self.head = new_node.next
        self.head.next = new_node
```

4. An AVL tree is a self-balancing binary search tree. However, in the following figure, the tree is unbalanced. Please identify the node that needs to be rotated and indicate whether it should be left-rotated, right-rotated, or both to restore the balance of the tree. Draw the modified tree after the rotation.



Part Four: Design and Analysis (36 marks in total)

(*Note that pseudo-code and even plain English are allowed*.)

1. Given a doubly linked list only with the head and rear, it is called "symmetric" if the values of the nodes on the left side are equal to the values of the nodes on the right side. For example, 1->2->3->4->3->2->1, 1->2->3->3->2->1, 3 and None are symmetric, while 1->2->3->4->5 is not symmetric. Please design a function *is_symmetric*() to check whether it is symmetric. (10 marks) (Hint: the function should return either True or False)

```
class LinkedList:
    def __init__(self):
        self._head = None
        self._rear = None

    def is_symmetric(self):
        pass # implement this
```

2. Given a binary search tree (BST) with a root node, please design an algorithm *count_nodes*() to count the total number of nodes in the BST. (10 marks) (Hint: use recursion to traverse the BST and count the nodes)

```
class BST:
    def __init__(self):
        self._root = None

    def count_nodes (self):
```

```
        pass # implement this
```

3. Given a singly unsorted linked list only with the head node. Please design an algorithm *range_of*(*start*, *end*) to get a array list storing all element between start and end (both are inclusive) . For example, a linked list is 9-> 2->8->10->15, the parameters *start* and *end* is 2 and 10 respectively, and the algorithm will output [2, 8, 10]. (10 marks)

```
class SinglyLinkedList:
    def __init__(self):
        self._head = None


    def range_of(self, start, end):
        pass # implement this
```

4. You can choose either A or B. (6 marks)

   A. Consider a transportation network where each node (an integer) represents a location, and each edge represents a road between two locations. Please design an algorithm *connections*() to return the number of locations that can be reached by location *l*.

```
class TransportationNetwork:
    def __init__(self, vertices):
        self.vertices = vertices
        self.adjacency_list = {vertex: [] for vertex in range(vertices)}

    def add_edge(self, u, v):
        self.adjacency_list[u].append(v)
        self.adjacency_list[v].append(u)

    def connections(self, l):
        pass # implement this
```

   B. Imagine you are implementing an algorithm that inserts a new node into a binary search tree (BST) while maintaining the BST's balance. The algorithm consists of traversing the BST to find the appropriate position for the new node and updating the parent node accordingly.

      a. What is the time complexity of this algorithm in terms of the number of nodes in the BST and analysis?

      b. If the BST becomes unbalanced due to some insertion operations, what would be the time complexity then?

      c. How can you optimize this algorithm to reduce its time complexity when inserting a new node into an unbalanced BST?